

x86-64 ASM Cheat sheet (1)

Data types

C type	Intel type	ASM	x86-64 size		gdb	
		Suffix	(byte)	(bits)	gdb type	x/ <code>
char	Byte	b	1	8b	byte	b or s
short	Word	w	2	16b	half-word	h
int	Double word	l	4	32b	word	w
long int	Quad word	q	8	64b	giant	g
char *	Quad word	q	8	64b	giant	a or g
float	Single precision	s	4	32b	word	wf
double	Double precision	d	8	64b	giant	gf
long double	Extended precision	t	10/16	80/128b		

Operand types

Type	Form	Operand value	Name
Immediate	$\$Imm$	Imm	Immediate
Register	E_a	$R[E_a]$	Register
Memory	Imm	$M[Imm]$	Absolute
Memory	(E_a)	$M[R[E_a]]$	Indirect
Memory	$Imm(E_b)$	$M[Imm + R[E_b]]$	Base + displacement
Memory	(E_b, E_i)	$M[R[E_b] + R[E_i]]$	Indexed
Memory	$Imm(E_b, E_i)$	$M[Imm + R[E_b] + R[E_i]]$	Indexed
Memory	$(, E_i, s)$	$M[R[E_i] \cdot s]$	Scaled indexed
Memory	$Imm(, E_i, s)$	$M[Imm + R[E_i] \cdot s]$	Scaled indexed
Memory	(E_b, E_i, s)	$M[R[E_b] + R[E_i] \cdot s]$	Scaled indexed
Memory	$Imm(E_b, E_i, s)$	$M[Imm + R[E_b] + R[E_i] \cdot s]$	Scaled indexed

scaling factor s must be either 1, 2, 4 or 8

General registers



Reg ID	quad word 63..0	double word 31..0	word 15..0	high byte 15..8	low byte 7..0	Procedure call arguments	Register saving
0	%rax	%eax	%ax	%ah	%al	Return value	
1	%rcx	%ecx	%cx	%ch	%cl	4th arg	
2	%rdx	%edx	%dx	%dh	%dl	3rd arg	
3	%rbx	%ebx	%bx	%bh	%bl		Callee
4	%rsp					Stack pointer	
5	%rbp	%ebp	%bp		%bpl		Callee
6	%rsi	%esi	%si		%sil	2nd arg	
7	%rdi	%edi	%di		%dil	1st arg	
8	%r8	%r8d	%r8w		%r8b	5th arg	
9	%r9	%r9d	%r9w		%r9b	6th arg	
10	%r10	%r10d	%r10w		%r10b		Caller
11	%r11	%r11d	%r11w		%r11b		Caller
12	%r12	%r12d	%r12w		%r12b		Callee
13	%r13	%r13d	%r13w		%r13b		Callee
14	%r14	%r14d	%r14w		%r14b		Callee
15	%r15	%r15d	%r15w		%r15b		Callee

ASM Instructions

Instruction	Params	Effect	Description
movb, movw, movl, movq	S, D	$D \leftarrow S$	Move
movzbw, ... movz<T> Types <T>: bw,bl,wl,bq,wq	S, R	$R \leftarrow \text{ZeroExtend}(S)$ bw = byte to word, etc...	Move with zero extension
movsbw, ... movs<T> Types <T>: bw,bl,wl,bq,wq,lq	S, R	$R \leftarrow \text{SignExtend}(S)$	Move with sign extension
cmtq		$\%rax \leftarrow \text{SignExtend}(\%eax)$	Sign-extend %eax register
cqto		$R[\%rdx]:R[\%rax] \leftarrow \text{SignExtend}(R[\%rax])$	Sign-extend to oct word
pushq	S	$R[\%rsp] \leftarrow R[\%rsp] - 8$ $M[R[\%rsp]] \leftarrow S$	Push quad word on stack
popq	D	$D \leftarrow M[R[\%rsp]]$ $R[\%rsp] \leftarrow R[\%rsp] + 8$	Pop quad word from stack
leaq	S, D	$D \leftarrow \&S$	Load effective address
inc, incq / dec, decq	D	$D \leftarrow D + 1 / D \leftarrow D - 1$, inc (32b), incq (64b)	Increment / Decrement
neg, negq / not, notq	D	$D \leftarrow -D / D \leftarrow \sim D$	Negate / Complement
add, addq / sub, subq	S, D	$D \leftarrow D + S / D \leftarrow D - S$	Add / Subtract
imul, imulq	S, D	$D \leftarrow D * S$	Multiply
xor, xorq / or, orq / and, andq	S, D	$D \leftarrow D \wedge S / D \leftarrow D \vee S / D \leftarrow D \& S$	Exclusive-or / Or / And
sal, salq, shl, shlq	k, D	$D \leftarrow D \ll k$	Left shift
sar, sarq	k, D	$D \leftarrow D \gg_A k$	Arithmetic right shift
shr, shrq	k, D	$D \leftarrow D \gg_L k$	Logical right shift
imulq	S	$R[\%rdx]:R[\%rax] \leftarrow S * R[\%rax]$	Signed full multiply
mulq	S	$R[\%rdx]:R[\%rax] \leftarrow S * R[\%rax]$	Unsigned full multiply
idivq	S	$R[\%rdx] \leftarrow R[\%rdx]:R[\%rax] \bmod S$ $R[\%rax] \leftarrow R[\%rdx]:R[\%rax] \div S$	Signed divide (remainder and quotient)
divq	S	$R[\%rdx] \leftarrow R[\%rdx]:R[\%rax] \bmod S$ $R[\%rax] \leftarrow R[\%rdx]:R[\%rax] \div S$	Unsigned divide (remainder and quotient)
cmpb, cmpw, cmpl, cmpq	S_2, S_1	Set CF, ZF, SF, OF based on $S_2 - S_1$	Compare
testb, testw, testl, testq	S_2, S_1	Set CF, ZF, SF, OF based on $S_2 \& S_1$	Test
setz, ... set<C>	D	Set D to 1 if condition <C> is true, to 0 else	Conditional set
Condition <C>: e / ne / s / ns same as: z / nz		equal (e), zeros(z) / non-equal (ne), non-zero(nz) / negative (s) / non-negative (ns)	ZF / SF codes
<C> = g / ge / l / le same as: nle / nl / nge / ng		greater > / greater or equal $\geq$ / less < / less or equal $\leq$	Signed comparison
<C> = a / ae / b / be same as: nbe / nb / nae / na		above > / above or equal $\geq$ / below < / below or equal $\leq$	Unsigned comparison
jmp	Label	Label may be replaced by *Operand	Direct / Indirect jump
jz, ... j<C>	Label	Jump to label if condition <C> is true	Conditional jump
cmovz, ... cmov<C>	S, R	$R \leftarrow S$ if condition <C> is true	Conditional move
call	Label	Label may be replaced by *Operand	Procedure call
ret			Return from call